



Golang Gin Backend Pertemuan 17

Membahas environment management, deployment server, dan systemd pada backend Golang Gin, lalu ditutup latihan praktik intermediate.



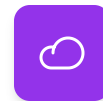
Juara Coding

June 2026

Tujuan Pembelajaran

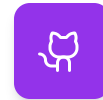


Peserta memahami environment, konfigurasi aman, deployment backend Gin, service systemd, serta operasi aplikasi di server Linux.



Memahami Environment

Membedakan development, staging, dan production untuk alur kerja aplikasi yang terstruktur.



Deployment dan Service

Melakukan deployment backend Gin ke Linux dan membuat service systemd untuk menjalankannya.



Monitoring dan Update

Menjalankan, memantau, dan memperbarui aplikasi secara aman dan konsisten.



No environment variables set

You haven't added any variables yet.

Add Variable

Konsep Environment Management

Mengatur konfigurasi aplikasi untuk kondisi berbeda tanpa mengubah kode utama, agar deployment tetap fleksibel, aman, dan konsisten.

Jenis Environment Backend

01
10

Development

Dipakai untuk coding dan debugging lokal dengan konfigurasi yang fleksibel.



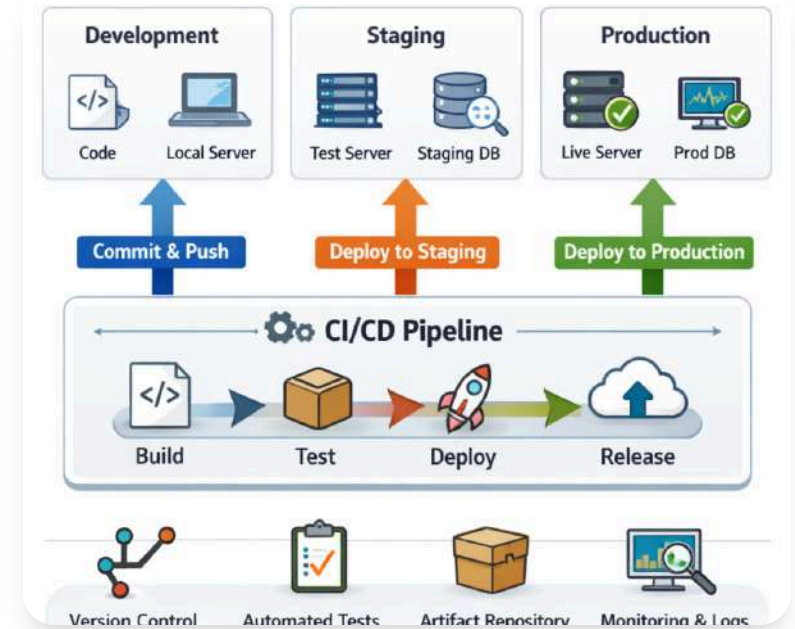
Staging

Digunakan untuk simulasi kondisi aplikasi sebelum rilis ke pengguna.



Production

Environment untuk pengguna akhir yang menuntut stabilitas, keamanan, dan monitoring tinggi.



Praktik Konfigurasi Gin

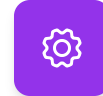
```
    "fmt"
}
func main() {
    port := os.Getenv('PORT')
    dbHost := os.Getenv("DB_HOST")
    dbUser := os.Getenv("DB_USER")
    dbPass := os.Getenv("DB_PASS")

    if port == "" {
        port = "8080"
    }

    router := gin.Default()

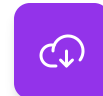
    router.GET("/ping", func(c*gin.Context) {
        c.JSON(200, gin.H{"message": "pong"})
    })
}
```

Pada aplikasi Gin, konfigurasi sebaiknya diambil dari environment variable agar mudah dipindahkan tanpa mengubah kode.



Gunakan Environment Variable

Simpan nilai seperti PORT dan DB_HOST di environment variable, bukan langsung di source code.



Mudahkan Deploy

Konfigurasi berbasis environment memudahkan pindah server tanpa mengubah logika aplikasi.

Contoh Variabel Environment

Variabel	Contoh Nilai
APP_ENV	production
PORT	8080
DB_HOST	127.0.0.1
DB_PORT	5432
DB_USER	appuser
JWT_SECRET	supersecretkey

Variabel seperti **APP_ENV**, **PORT**, dan kredensial database sebaiknya dipisahkan dari kode agar lebih aman serta lebih mudah dikelola saat deploy dan maintenance.

Strategi Penyimpanan Konfigurasi



01 Environment Variable atau .env

Konfigurasi dapat disimpan lewat environment variable sistem atau file .env saat tahap development.

02 Production di Level Sistem

Untuk server production, gunakan variable pada level sistem atau service manager agar lebih aman.

03 Hindari Kredensial di Repository

Jangan menyimpan kredensial sensitif di repository karena berisiko bocor dan disalahgunakan.

```
Loading configuration...
```

```
Environment Validation: None
```

```
ERROR: Required environment variables are not set!
```

```
MISSING ENV: DATABASE_URL
```

```
MISSING ENV: API_KEY
```

```
MISSING ENV: SECRET_TOKEN
```

```
Application startup failed!
```

Loader Config Sederhana di Go

Package config membaca semua environment variable saat startup dan memvalidasi nilai penting agar aplikasi gagal lebih awal.


```
4      "fmt"
5      "os"
6  }
7
6  func main() {
9      os.Setenv("DB_USER", "admin")
10     os.Setenv("DB_PASS", "password123")
11     os.Setenv("API_KEY", "abdef123456")
12
13     user := os.Getenv("DB_USER")
14     pass := os.Getenv("DB_PASS")
13     apiKey := os.Getenv("API_KEY")
20
17     fmt.Println("DB User:", user)
18     fmt.Println("DB Pass", pass)
19     fmt.Println("API Key", apiKey)
10 }
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

None
\$

Contoh Baca Environment

Kode Go membaca variabel *PORT*, memakai default *8080*, lalu menjalankan server Gin dengan endpoint */health*.

Validasi Konfigurasi Penting



Konfigurasi sensitif seperti `DB_PASSWORD` dan `JWT_SECRET` harus wajib diisi agar aplikasi tidak berjalan dalam kondisi tidak aman.



Tanpa Default Sensitif

Nilai seperti password dan secret tidak boleh memakai default agar tidak lolos tanpa sengaja.



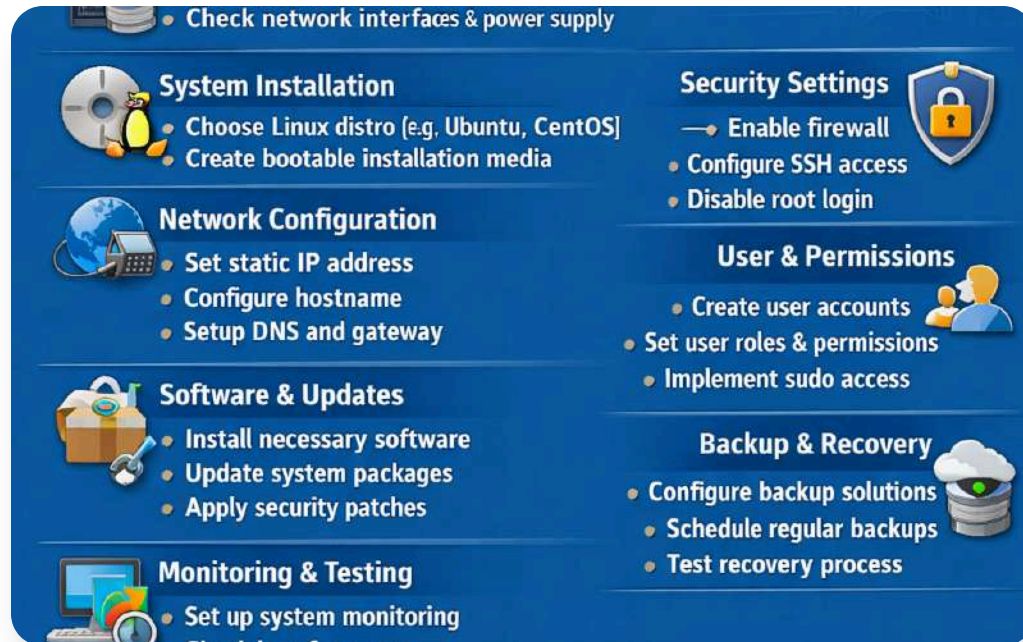
Gagal Saat Startup

Aplikasi harus berhenti saat konfigurasi wajib kosong untuk mencegah risiko lebih lanjut.



Cegah Masalah Produksi

Validasi awal membantu menemukan konfigurasi salah sebelum sulit dideteksi di production.



Persiapan Deployment

Siapkan akses server, runtime, port, firewall, binary, dan direktori rilis.

Alur Deployment Sederhana

01 Build dan Upload Binary

Kompilasi binary Go lalu unggah ke server sebagai artefak utama aplikasi.

03 Daftarkan Service dan Verifikasi

Daftarkan ke systemd lalu cek endpoint seperti */health* untuk memastikan aplikasi aktif dan siap digunakan.

02 Konfigurasi dan Uji Awal

Salin file konfigurasi, jalankan aplikasi, lalu lakukan uji awal untuk memastikan proses berjalan baik.

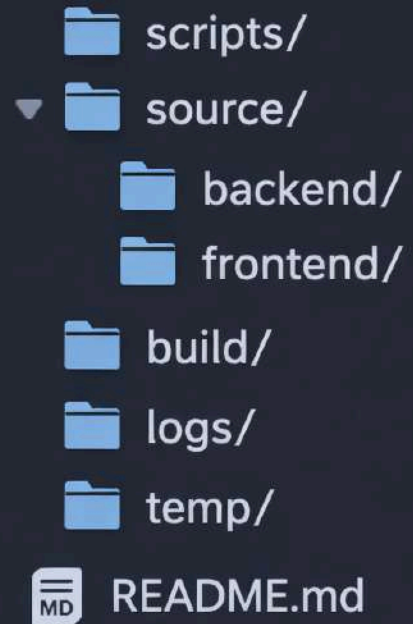
```
1. Updating System Packages...
2. Installing Dependencies...
3. Configuring Firewall Settings...
4. Setting Up Environment Variables...
5. Deploying Application...
6. Starting Services...
7. Verifying Deployment...

Updating packages... [OK]
Installing dependencies... [OK]
Configuring firewall... [OK]
Setting environment variables... [OK]
Deploying application... [DONE]
```



Build Binary untuk Server

Go menghasilkan binary mandiri, jadi aplikasi mudah dibangun di lokal atau CI lalu dikirim ke server sebagai satu executable.



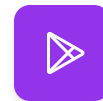
Penempatan File di Server

Tempatkan binary di direktori aplikasi seperti **/opt/myapp** dan pisahkan file konfigurasi agar maintenance lebih mudah.

Menjalankan Aplikasi Manual

```
Starting application...
Environment: None
[INFO] Loading configuration...
[INFO] Connecting to database...
      Database connection failed !
[ERROR] Unable to connect to the database!
[INFO] Retrying connection (1/3)...
      Connection failed.
[INFO] Retrying connection (2/3)...
      Connection failed.
[INFO] Retrying connection (3/3)...
      Connection failed.
[ERROR] Application startup failed. Exiting.
```

Jalankan binary secara manual lebih dulu untuk memastikan aplikasi benar-benar bisa start sebelum dibuat sebagai service systemd.



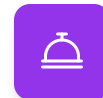
Validasi Startup

Menjalankan binary langsung membantu memastikan aplikasi bisa start normal tanpa error awal.



Cek Masalah Dasar

Tahap ini membantu menemukan salah konfigurasi, port bentrok, atau koneksi database gagal.



Lanjut ke systemd

Jika sudah stabil saat manual, otomatisasi service bisa dilakukan dengan lebih aman.



Apa Itu systemd

systemd mengelola service Linux agar aktif otomatis, mudah dipantau, dan bisa direstart saat gagal.

Manfaat Menggunakan systemd



Auto-start saat boot

Layanan backend berjalan otomatis saat server dinyalakan tanpa langkah manual.



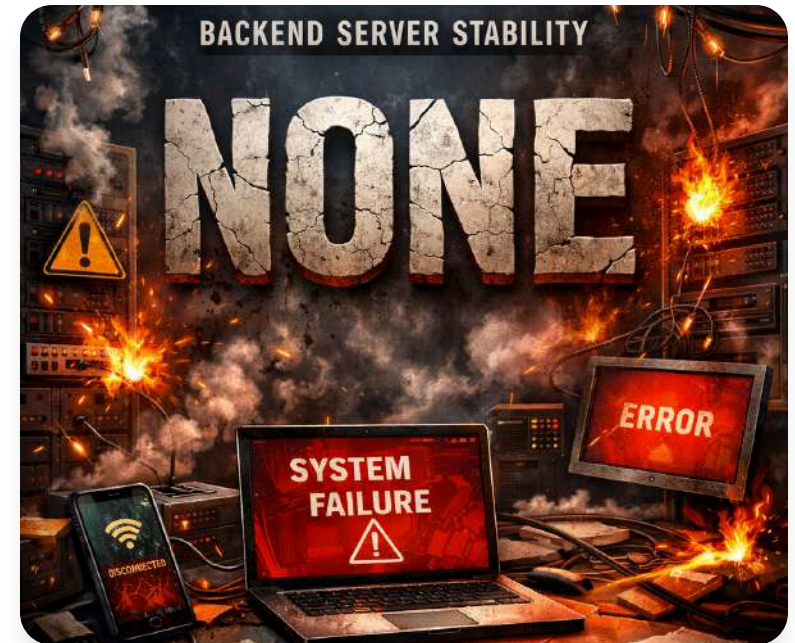
Restart otomatis

Service aktif kembali saat crash sehingga ketersediaan aplikasi lebih terjaga.



Kontrol dan logging

Pengelolaan service lebih standar dan terintegrasi dengan logging untuk monitoring production.



Komponen Dasar File Service

```
[Unit]
Description=My Custom Service
After=network.target
Wants=other.service

[Service]
Type=simple
ExecStart=/usr/bin/myservice --option
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
User=myuser
WorkingDirectory=/var/www/app
Environment "ENV_VAR=value"
RestartSec=10s
```

01 Bagian Unit

Berisi deskripsi layanan dan urutan dependensi agar systemd memahami hubungan antar layanan.

02 Bagian Service

Menentukan perintah utama, cara menjalankan aplikasi, serta perilaku proses saat service aktif.

03 Bagian Install

Mengatur target aktivasi sehingga service dapat diaktifkan otomatis saat proses boot sistem.

```
Description=Backend Application
```

```
After=network.target
```

```
[Service]
```

```
User=backenduser
```

```
WorkingDirectory=/opt/backend
```

```
ExecStart=/usr/bin/python3/opt/backend/app.py
```

```
Restart=always
```

```
RestartSec=5
```

```
[Install]
```

Contoh File Service systemd

File service systemd ini menjalankan aplikasi backend Gin dengan direktori kerja, perintah start, restart otomatis, dan variabel environment.

Langkah Mengaktifkan Service

```
root@server:~# systemctl status apache2
apache2.service - The Apache HTTP Server
Loaded (/lib/systemd/system/apache2.service; enabled; vendor preset: enabled)
Active: active (running) Since Tue 2024-04-24 10:15:32 UTC; 2min ago
Docs: https://httpd.apache.org/docs/2.4/
Main PID: 1234 (apache2)
Tasks: 8 (limit: 512)
Memory: 15.3M
CGroup: /systemd/system/apache2.service
├─1234 /usr/sbin/apache2 -k start
├─1256 /usr/sbin/apache2 -k start
└─1270 /usr/sbin/apache2 -k start
24-10-15:22 myserver systemd[1]: Started The Apache HTTP Server.
```

01 Reload konfigurasi

Jalankan *daemon-reload* agar perubahan file service dikenali oleh sistem dengan benar.

02 Aktifkan dan jalankan

Gunakan *enable* lalu *start* untuk mengaktifkan service saat boot dan menjalankannya sekarang.

03 Periksa status

Cek *status* untuk memastikan service berjalan normal dan tidak ada error pada proses startup.



Monitoring dan Log Service

Pantau status service dan log untuk mendeteksi restart berulang, gagal binding port, atau masalah pembacaan environment variable.

Update dan Restart Aplikasi

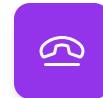


Update dilakukan dengan mengganti binary lama, me-restart service, lalu memverifikasi endpoint agar rilis tetap aman.



Ganti Binary

Gunakan binary versi baru untuk menggantikan binary lama secara terkontrol.



Restart dan Verifikasi

Restart service lalu cek endpoint untuk memastikan aplikasi berjalan normal.



Siapkan Rollback

Simpan versi sebelumnya agar pemulihan dapat dilakukan cepat saat ada masalah.

Praktik Keamanan Deployment



Jangan Simpan Secret di Kode

Simpan kredensial di secret manager atau environment variable agar tidak bocor lewat source code.



Batasi Hak Akses

Atur permission file binary dan konfigurasi seperlunya untuk mengurangi penyalahgunaan akses.



Pisahkan Environment

Gunakan user khusus untuk service dan bedakan production dari development demi keamanan operasional.



Kesalahan Umum yang Harus Dihindari



Hardcode password di kode

Simpan kredensial di environment variable atau secret manager, bukan langsung di source code.



Aplikasi gagal di production

Kesalahan kecil pada konfigurasi server sering membuat service tidak bisa berjalan dengan benar.

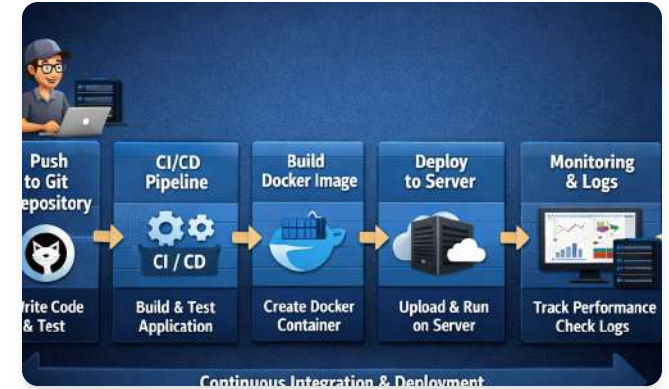


Konfigurasi dan deploy tidak dicek

Pastikan environment variable, path ExecStart, pengujian binary, dan status service diperiksa setelah deploy.



Rangkuman Alur Kerja



01 Siapkan aplikasi dan konfigurasi

Mulai dengan menyiapkan aplikasi Gin lalu membaca konfigurasi dari environment agar setup tetap konsisten.

03 Aktifkan service dan monitor

Buat file service systemd, aktifkan service, kemudian pantau status dan log agar deployment lebih rapi.

02 Build, unggah, dan uji

Build binary, unggah ke server, lalu lakukan uji manual untuk memastikan aplikasi berjalan sesuai harapan.



SSH into the server

```
ssh user@your-server-ip
```



Update & install packages:

```
sudo apt update  
sudo apt install golang
```

3. Build & Configure App



Build the app:
go build main.go

- Set environment variables
- Create a systemd service file
(`/etc/systemd/system/ginapp.service`)

4. Start & Enable Service



Start the service:

```
sudo systemctl start ginapp
```



Enable on boot:

```
sudo systemctl enable ginapp
```



Check status:

```
sudo systemctl status ginapp
```



Access Your API

Latihan Praktik Intermediate

Buat backend Gin dengan endpoint `/health` dan `/profile`, baca `APP_ENV`, `PORT`, `JWT_SECRET`, lalu deploy ke Linux sebagai service *systemd*.

Latihan Praktik: Kriteria

Aspek	Kriteria
Port aplikasi	Berjalan pada port dari environment variable
Endpoint health	/health mengembalikan respons sukses
Service systemd	File service berfungsi dengan benar
Operasi service	Dapat enable dan start tanpa error
Verifikasi deployment	Menunjukkan status service dan uji endpoint

Tabel ini merangkum kriteria utama keberhasilan latihan praktik intermediate setelah proses deployment selesai.

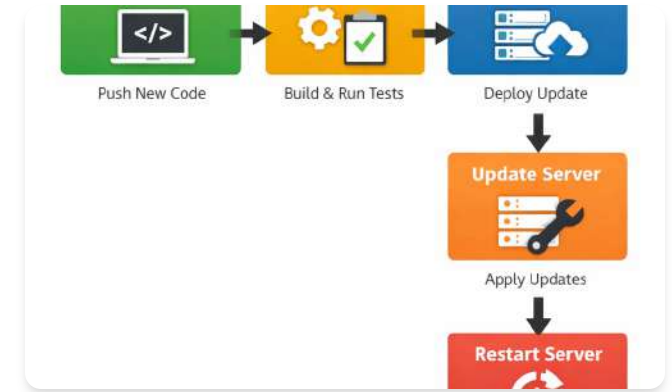
Latihan Praktik Intermediate

01 Validasi JWT_SECRET

Tambahkan validasi agar aplikasi gagal startup bila JWT_SECRET kosong atau tidak diatur.

03 Simulasi Update Aplikasi

Ganti binary, restart service, lalu dokumentasikan langkah deployment singkat agar mudah diulang tim.



02 Struktur Deployment

Buat susunan folder deployment yang rapi untuk binary, config, log, dan file service.

Penutup

“

"Environment management, deployment server, dan systemd adalah fondasi penting untuk membawa aplikasi Gin dari lokal ke production."

Ringkasan

Deployment in Progress...



Deploying Backend Services...